Lecture Notes in Electrical Engineering

Volume 75

Aimé Lay-Ekuakille and
Subhas Chandra Mukhopadhyay (Eds.)

Wearable and Autonomous Biomedical Devices and Systems for Smart Environment

Issues and Characterization

Springer

Aimé Lay-Ekuakille, Guest Editor
Dipartimento d'Ingegneria dell'innovazione
University of Salento
Lecce, Italy
E-mail: aime.lay.ekuakille@unisalento.it

Subhas Chandra Mukhopadhyay, Guest Editor
School of Engineering and Advanced Technology (SEAT),
Massey University (Turitea Campus)
Palmerston North, New Zealand
E-mail: S.C.Mukhopadhyay@massey.ac.nz

ISBN 978-3-642-15686-1 e-ISBN 978-3-642-15687-8

DOI 10.1007/978-3-642-15687-8

Library of Congress Control Number: 2010934865

Typeset & Coverdesign: Scientific Publishing Services Pvt. Ltd., Chennai, India.

Printed on acid-free paper

9 8 7 6 5 4 3 2 1

springer.com

Guest Editorial

The human body, with its articulations, is a complex physical bio-system that needs a constant check-up not only for remedial reasons (bio-psycho-physical degradation) but also for compensating limitations imposed by impairments and for improving performances in particular situations. Beyond the above reasons there are others related to the use of external non-invasive biomedical devices and instrumentation necessary for monitoring physiological parameters and for allowing diagnosis. Further issues connecting previous aspects regard technologies of managing intrinsic and extrinsic phenomena that surround the human body according to information processing. The quick growth of smart sensors, low-power embedded devices and new materials from physical and technological viewpoints, represents a stimulus to provide new applications for the human body well-being.

Hence, continuous advances in wearable and portable sensors for many applications and especially for biomedical issues have been sufficient reasons that are encouraged the necessity to collect variety of experience in this field; thanks to that, a new book has been prepared which title is “Wearable and autonomous biomedical devices and systems: New issues and characterization” and included in the book series of “Lecture Notes in Electrical Engineering”. This special issue dedicated to wearable and autonomous systems, including devices, offers to variety of users, namely, master degree students, researchers and practitioners, an opportunity of a dedicated and a deep approach in order to improve their knowledge in this specific field. The special issue draws the attention about interesting aspects, as for instance, advanced wearable sensors for enabling applications, solutions for arthritic patients in their limited and conditioned movements, wearable gate analysis, energy harvesting, physiological parameter monitoring, communication, pathology detection , etc..

The book, on one hand, illustrates theoretical aspects and applications, and it displays new criteria in characterizing devices and systems, at the other hand. Characterization is a key issue since it allows to know the performance of devices and systems described in the book by showing some statistics and result representation. The book contains 20 contributions from colleagues working in the topic and under different approaches and aspects; these co-ordinated approaches are the true richness of the book. The editors gracefully thank the contributors for their single papers included in this special issue. The editors hope this special

issue will be a good omen for all neophytes who approach the topic for the first time but also for readers with experience who can breathe fresh life into their research.

Aimé Lay-Ekuakille, Guest Editor
Dipartimento d'Ingegneria dell'innovazione
University of Salento
Lecce, Italy
aime.lay.ekuakille@unisalento.it

Subhas Chandra Mukhopadhyay, Guest Editor
School of Engineering and Advanced Technology (SEAT),
Massey University (Turitea Campus)
Palmerston North, New Zealand
S.C.Mukhopadhyay@massey.ac.nz

Dr. Subhas Chandra Mukhopadhyay graduated from the Department of Electrical Engineering, Jadavpur University, Calcutta, India in 1987 with a Gold medal and received the Master of Electrical Engineering degree from Indian Institute of Science, Bangalore, India in 1989. He obtained the PhD (Eng.) degree from Jadavpur University, India in 1994 and Doctor of Engineering degree from Kanazawa University, Japan in 2000.

During 1989–90 he worked almost 2 years in the research and development department of Crompton Greaves Ltd., India. In 1990 he joined as a Lecturer in the Electrical Engineering department, Jadavpur University, India and was promoted to Senior Lecturer of the same department in 1995.

Obtaining Monbusho fellowship he went to Japan in 1995. He worked with Kanazawa University, Japan as researcher and Assistant professor till September 2000.

In September 2000 he joined as Senior Lecturer in the Institute of Information Sciences and Technology, Massey University, New Zealand where he is working currently as an Associate professor. His fields of interest include Sensors and Sensing Technology, Electromagnetics, control, electrical machines and numerical field calculation etc.

He has authored over 200 papers in different international journals and conferences, edited nine conference proceedings. He has also edited six special issues of international journals as guest editor and seven books with Springer-Verlag.

He is a Fellow of IET (UK), a senior member of IEEE (USA), an associate editor of IEEE Sensors journal and IEEE Transactions on Instrumentation and Measurements. He is in the editorial board of e-Journal on Non-Destructive Testing, Sensors and Transducers, Transactions on Systems, Signals and Devices (TSSD), Journal on the Patents on Electrical Engineering, Journal of Sensors. He is in the technical programme committee of IEEE Sensors conference, IEEE IMTC conference and IEEE DELTA conference. He was the Technical Programme Chair of ICARA 2004 and ICARA 2006. He was the General chair of ICST 2005, ICST 2007. He has organized the IEEE Sensors conference 2008 at Lecce, Italy as General Co-chair and the IEEE Sensors conference 2009 at Christchurch, New Zealand as General Chair during October 25-28, 2009.

Aimé Lay-Ekuakille has a Master Degree in Electronic Engineering, a Master Degree in Clinical Engineering, a Ph.D in Electronic Engineering from Polytechnic of Bari. He has been technical manager of different private companies in the field of: Industrial plants, Environment Measurements, Nuclear and Biomedical Measurements; he was director of a Health & Environment municipal Department. He has been a technical advisor of Italian government for high risk plants. From 1993 up to 2001, he was adjunct professor of Measurements and control systems in the University of Calabria, University of Basilicata and Polytechnic of Bari. He joined the Department of Innovation Engineering, University of Salento, in september 2000 in the Measurement & Instrumentation Group. Since 2003, he became the leader of the scientific group; hence, he is the co-ordinator of Measurement and Instrumentation Lab in Lecce. He has been appointed as UE Commission senior expert for FP-VI (2005–2010). He is the Editor-in-Chief of the International Journal of Measurement Technologies and Engineering Instrumentation - IJMTIE (IGI-Global, USA). He is still: chair of IEEE-sponsored SCI/SSD Conference, member of Transactions on SSD and Sensors & Transducers Journal editorial board. He is Associate Editor of the International Journal on Smart Sensing and Intelligent Systems. He is currently organizing the next ICST2010 in Lecce, Italy. He is a member of the following boards and TCs: Association of the Italian Group of Electrical and Electronic Measurements (GMEE), SPIE, IMEKO TC19 Environmental Measurements, IEEE, IEEE TC-25 Medical and Biological Measurements Subcommittee on Objective Blood Pressure Measurement, IEEE-EMBS TC on Wearable Biomedical Sensors & System and included in different IPCs of conferences. Aimé Lay-Ekuakille is a scientific co-ordinator of different international projects. He has authored and co-authored more than 110 papers published in international journals, books and conference proceedings. His main researches regard Environmental and Biomedical instrumentation & measurements and, measurements for renewable energy.

Contents

Design and Integration of Fall and Mobility Monitors in Health Monitoring Platforms

Pepijn van de Ven, Alan Bourke, John Nelson, and Hugh O'Brien

University of Limerick, Limerick, Ireland
pepijn.vandeven@ul.ie

1 Introduction

This chapter discusses the design and integration of fall and mobility sensor platforms for mobile and remote health signs monitoring. With a steadily increasing elderly population in Europe [1] and indeed throughout significant parts of the rest of the world, health services for elderly people are placing a growing strain on national health budgets [2] and the availability of nursing and care taker staff. Additionally, perhaps due to the advent of technology in general and changing social relations in society, there is an ever-increasing wish amongst our elderly citizens to live independently and be mobile for as long as possible. Recent advances in telecommunications, medical devices and technology in the home environment have enabled elderly people to live independently for longer than ever before. However, integrated systems targeting the monitoring of these elders' health in the home environment are at best scarce. The use of these systems would enable medical practitioners to perform an early diagnosis of potential issues, which in turn would result in a better chance of full recovery and lower hospitalization costs. A common problem encountered by elderly people is a reduction in their mobility. It is estimated that one in three elderly people 65 years or older in the UK experience a serious fall each year [3]. The results of a fall can be dramatic, leading to long hospitalization and, not seldom, death as a direct or indirect consequence of the fall [3, 4]. The resulting emotional and financial cost to individuals and society is significant. In a recently published paper [5], Gannon et al. estimated the annual cost of falls amongst elderly citizens in Ireland, which has a population of 4 million, to be EUR 404 Million. In the United States the cost of falls for people aged 65 and over were estimated at over $19 billion for the year 2000[6]. People experiencing reduced mobility, in particular the elderly, can greatly benefit from having a fall and mobility sensor for several reasons. In the event of a fall, it is of great importance that help is requested immediately to prevent a so-called long-lie event. All too often, elderly citizens spend hours or longer on their own after having fallen. Getting help immediately has shown to lower the risk of hospitalization by 26%, and

A. Lay-Ekuakille et al. (Eds.): Wearable and Autonomous Systems, LNEE 75, pp. 1–29.
springerlink.com

death as a result of the fall by 80% [7]. To prevent falls from happening in the first place, it is important that elderly people (or other groups of citizens at higher risk of falling) and their health providers have access to an early warning system that indicates an increased risk of falling. Mobility monitoring is a proven means of predicting fall risk and continuous assessment of an elderly person's mobility is therefore crucial. Current solutions focus on providing a stand-alone solution to fall and mobility monitoring; the sensors may share interfaces with other health signs sensors, but the measurements are not correlated with other health signs or used to optimize the measurement of those other health signs. For example, if an elderly person's blood pressure or heart rate is measured just after vigorous exercise (walking stairs could easily fall in this category), results may be interpreted as alarming if there is no information regarding the recent activity. The importance of including fall and mobility monitors in mobile health monitoring systems is evident. However, the close and careful integration of the functionality added by the fall and mobility monitor with all the other sensor functionality is extremely important to take full benefit of the extra information. This chapter discusses such an integration effort. After an overview of existing fall and mobility algorithms and sensors in section 2, fall and mobility sensor hardware, made in the University of Limerick, is presented in section 3. The software implementing the fall and mobility monitoring algorithms is discussed in section 4 after which a generic sensor interface for use on resource-constrained mobile platforms is discussed in section 5. After this description of fall and mobility monitoring hardware and software, section 6 discusses trials performed with the fall and mobility sensors as part of a large project funded by the European Commission.

2 Methods of Fall Detection

In 1987 the Kellogg international working group on the prevention of falls in the elderly defined a fall as 'unintentionally coming to ground, or some lower level not as a consequence of sustaining a violent blow, loss of consciousness, sudden onset of paralysis as in stroke or an epileptic seizure' [8]. This definition has been used in many research studies, and sometimes extended to include falls resulting from dizziness, syncope and consequences of an epileptic fit or cardiovascular collapse. The World Health Organization has defined a fall as 'an event, which results in a person coming to rest inadvertently on the ground or some other lower level.' Thus fall detection systems should be able to detect falling onto the ground in an unexpected and uncontrolled way, whatever the cause of this accidental collapse.

2.1 Fall Detection Algorithms

The end of a fall may be characterized by an impact (which causes the physical damage) and by near horizontal orientation of the faller. Fall-detection

systems must detect one if not both of these characteristics associated with a fall[4]. The most obvious method of fall detection involves the sensing of the shock received by the body upon impact. The preferred sensor for this purpose is the accelerometer. Attached to the body it is used to measure deceleration of the body when it is arrested by the ground following a fall. This method is used in many subject-worn fall-detection systems [9, 10, 11, 12, 13, 14]. Also included in some fall-detection systems is the detection of the orientation of the subject [11, 12, 14, 15, 16, 17]. The body orientation characteristic is used as a confirmation that the person has fallen and not just bumped into someone or something. The measurement of the body orientation is accomplished using tilt switches incorporated into the subject-worn fall-detection device, or through extraction of the orientation specific part of the accelerometer signal.

Academic research efforts

In 1991 Lord and Colvin [9] reported some early work on fall detection using a tri-axial accelerometer. Their work described a data-logging unit to measure accelerometer signals for later analysis. Williams in 1998 [11] described the concept of an autonomous device, carried on the belt, which featured a piezoelectric sensor to detect the impact when the person hit the ground, as well as a mercury tilt switch to detect when the person was horizontal. The detection thresholds for impact were determined by attaching the sensors to a mannequin, which fell onto a wooden floor. This work was later continued by Doughty [18], who patented a two-stage fall-detection system and algorithm and a commercial version of this system was developed and marketed by the company Tunstall (Whitley lodge, Yorkshire, England). Doughty's fall-detector wakes up from the sleep state when a strong impact is detected. Then a second sensor estimates the body orientation of the wearer and determines whether the subject is in a lying state for a set period of time. If both these decision rules are satisfied an alarm is raised. Wu [19] from the University of Vermont-USA showed, using video analysis of markers placed on the subject, that vertical and horizontal speeds are 3 times higher during a fall than for any other controlled normal activity. Wu also showed that both speeds are of the same amplitude at the time of the fall whereas they are strongly dissimilar during 30 normal activities. This work inspired Nait-Charif [20] to detect falls from computer vision techniques using particle filters to track movement of the subject's head. Successful fall detection results were obtained through the use of fixed thresholds on vertical and longitudinal velocities. Tamura [17] proposed an ambulatory monitor that records falling time over a long period. When the subject is falling a photo-interrupter outputs a trigger signal and the microprocessor records the falling time. In a preliminary study, the system was tested for normal adults and hemiplegic patients, and operated without any trouble. However, fundamentally the sensor is just a tilt switch and thus false-positives occurred when subjects lay down or rode a

bike. Noury [15, 16, 14], from the University of Grenoble-France, designed an autonomous sensor module, attached under the armpit, which comprised accelerometers, inclinometers and a vibration sensor. The sensor module detected when the velocity of the movement exceeded a specific threshold, the sequence from a vertical posture to the lying posture, as well as the absence of movements after the fall. The device was first tested on 5 persons who performed 15 fall scenarios 5 times and was shown to achieve a sensitivity and specificity close to 85%. Demongeot [21] developed a 'tele-monitoring' system to remotely detect the physical and physiological health aspects of elderly people living at home. The system also incorporated a fall detection feature. A fall was detected when a combination of, subject-worn sensors and local environment or embedded sensors triggered simultaneously. The sensors include infrared-volumetric sensors, magnetic door contacts and the accelerometer readings (attached to the subject). The combination of information from all of these sensors was used to indicate that a fall might have occurred. A heart-rate monitor was used to ensure that the subject wore the device. This fall-detection system is part of an over-all 'health-smart-home' and is an example of a secondary-fall detection system. A fall is detected when certain criteria are fulfilled and the system detects a deviation from normal habit in the persons activity. As a result, a fall may not be detected straightaway unlike primary fall-detection systems. Degen [22] developed a fall detector wrist watch which obtained a 65% sensitivity and 100% specificity. The device consists of a tri-axial accelerometer and uses three thresholds to distinguish falls from Activities of Daily Living (ADL). If all three exceed their thresholds, a fall is detected. Three subjects performed a number of falls. Results show a 100% detection of forward falls, 58% detection of backwards falls and 45% detection of sideways falls. One subject then wore the device for 48 hours in which time no false-positives occurred. Karantonis [23] developed a subject worn fall-detector and mobility monitor which incorporated a single tri-axial accelerometer. The algorithm for the fall-detection algorithm operates on the device itself and is not processed in a remote location. If at least two consecutive peaks in the signal vector magnitude exceed a threshold of 1.8g a possible fall flag is set. The subject is then monitored continuously for 60 seconds after the possible fall and after an initial 5-second settling period. If the subject is motionless during this time. Then the event is reclassified as a fall. This would require immediate attention. The algorithm was tested using five young healthy subjects. Each performed three different fall types and a number of sitting, lying and walking ADL at different speeds. A number of the lying activities were incorrectly detected as possible falls. The overall sensitivity of the fall detection algorithm was 95.6%. Prado [13] developed an intelligent 4-axis accelerometer unit (IAU) worn like a patch, fixed to the back at the height of the sacrum. Further evaluation by Diaz [10] in a laboratory study carried out over 8 volunteers, showed that the device was able to distinguish true fall events from normal activities like fast walking or going up/downstairs. The device produced a 100% sensitivity and

92.5% specificity. Kangas et al. [24] evaluated different low-complexity fall detection algorithms, using tri-axial accelerometers attached at the waist, wrist, and head. Fall data was recorded from three middle-aged subjects who performed nine intentional fall types in three directions (forward, backward, and lateral). High sensitivities ($97\% - 98\%$) and specificities (100%) were achieved in this study with quite low-complexity algorithms, however only a small subject group ($n¡=3$) performing very few scripted ADL was used and no long-term unsupervised testing was performed. The same researchers also continued the evaluation of the algorithms on data recorded by a group of 20 middle-aged (40-65 years old) volunteers performing falls (six different falls each performed twice) and four scripted ADL, also these same ADL were also recorded from 21 older people (aged 58-98 years) from a residential care unit. Chao et al. [25] recorded tri-axial accelerometer data from the chest and waist of seven young healthy male subjects as they performed falls as well as functional normal and dynamic activities including jumping. They examined the use of acceleration cross-product and magnitude as well as post fall posture and developed and tested algorithms of varying complexity. The various algorithms were analysed under the following categories; threshold selection, sensor placement and post-fall posture. Results showed that a fall detection algorithm based on the data from the chest shows better global accuracy. With post-fall posture detection leading to lower false alarm ratios.

2.2 Sensor Location

In several studies performed in the area of falls, fractures of the hip and proximal femur were identified as typical injuries sustained during a fall-event [26, 27, 28]. Indeed this is a large area of concern, with hip fractures being the most common and frequent of all serious injuries sustained by the elderly population following a fall [9]. But as statistics have shown, injuries from falls are not confined to this region. A large amount of wrist, radius and elbow fractures as well as shoulder and head injuries occur from falls also [29, 30]. According to Veltink et al. [31] to provide general kinematic data from a subject using subject-worn sensors, sensors should be easily mountable on the body and stay reliably in place at all times. Sensors should not be uncomfortable to the subject and not impede their activity of daily living, so they should not cross any joints and only require short cables. Thus to provide comprehensive detection of all collisions to all major body parts due to fall-events, the proposed sensors should measure the impact occurring to both the upper and lower extremities during a fall-event, as well as being non-intrusive and not cumbersome. Most of the fall detection devices and systems that will be discussed use one or more sensors placed on the upper part of the body, at the upper trunk and at the pelvic region. Doughty et al. [18] placed piezoelectric shock sensors on the trunk (chest), thigh, waist and wrist, but later confined the sensor locations to the waist and chest. One of the major issues with fall detection devices is the lack of compliance with

wearing these sensors. The idea of a wrist worn fall detector is highly suitable as the sensor would replace the already acceptable wrist watch. A fall-detector placed at the wrist was investigated as a possible location by Doughty et al. [18] and Degen et al. [22]. According to the authors the wrist would be a very appealing location as 'A fall detector in the form of a wrist watch will not feel alien to the wearer', and it could be worn comfortably during high risk fall situations like showering or going to bed. However, according to the same authors 'The major disadvantage of this solution is the complexity of the fall detection algorithm in detecting impact from a fall. The arm can move and rotate, thus has six degrees of freedom in its movement.' Doughty et al.[18] also conceded that the results obtained with a wrist-worn sensor for different fall types varied too widely for a fall detector to be placed there.

2.3 Sensor Attachment

Most accelerometer based fall detection and mobility monitoring systems located on the trunk have used either chest straps [31, 12, 32, 33, 34, 35] or patches [10, 13, 36]. These methods of attachment have significant draw backs:

- They normally result in systems which are uncomfortable to wear
- The system is not easily donned
- Sensors can be easily placed incorrectly
- More than one person may be required to attache the sensors

Fall detection systems incorporated in wearable garments eliminate many of these problems. A number of garments that incorporate fall detection sensors exist and are detailed here.

- MEMSWear smart shirt
  The MEMSWear smart shirt was developed by Tay [37]. It incorporates a tri-axial accelerometer into the shoulder. By setting a threshold of 4.8g on the summation of the peak values from the acceleration signals, separation between gathered fall and ADL data is achieved, however these peaks do not occur at the same time and it is not clear what time window, if any, is used.
- VTAMN
  Garment based mobility monitoring has been attempted by Noury [38] who developed a smart vest for ambulatory remote monitoring of physiological parameters and activity. A fall detection system is incorporated into the vest under the left-arm, it consists of a bi-axial accelerometer and a micro-controller embedded on a flexible electronic board. The fall detection algorithm is based on research also performed by Noury [15, 16, 14]. In clinical trials where the vest was worn by 3 healthy adults for 4 days the VTAMN was found easy to put on and the thermal comfort was acceptable, even with high ambient temperatures (36°). The vest delivers physiological information on the subject including heart rate, respiratory

rate and temperature as well as environmental parameters (ambient temperature) and can be used to perform fall detection.

- Lifeshirt by Vivometrics
  The lifeshirt developed by Vivometrics (www.vivometrics.com) is a lightweight, machine washable shirt with embedded sensors. The respiratory sensors are woven into the shirt around the wearer's chest and abdomen. A single channel ECG measures heart rate, and a three axis accelerometer located at the sternum, records posture and activity. Optional peripheral devices measure EEG/EOG, periodic leg movement, temperature, end tidal CO2, blood oxygen saturation, blood pressure and coughing.
- The smartshirt by sensatex
  The sensatex smartshirt (www.sensatex.com) is a seamless light breathable, nylon fabric with fully integrated conductive fibres, creating connectivity to acquire and transfer physiological signals. The shirt is designed to monitor an individual's heart rate, respiration, body temperature and movement wirelessly and remotely. The SmartShirt collects physiological signals from the wearer's body which are converted to digital signals using a small personal controller and sent wirelessly to a base station through either Bluetooth or ZigBee wireless technology.

3 Fall Sensor Hardware

This section describes the hardware used by the authors in several projects. In these projects the objectives are to accurately identify falls, monitor mobility or perform both. The fall and mobility sensor is based on a Freescale tri-axial accelerometer controlled by a Texas Instruments low power microprocessor. The microprocessor is used for AD conversion of the accelerometer data, extraction of mobility and fall data, storage of the data on a microSD card and communication with a PC or mobile phone. A block schematic of the fall and mobility sensor system is shown in figure 1. The Bluetooth module used in this design is the Roving Networks RN-24.

3.1 Microprocessor

The Texas Instruments MSP430F1611 was chosen for its low power consumption, sizeable memory (48kB flash with 256B user flash and 10kB RAM) and relatively high speed (8MHz). Using the IAR workbench or similar software, the MSP430 can be programmed using C/C++ or assembler. The MSP430F1611 is equipped with a 16 bit Reduced Instruction Set Computer (RISC) processor, 3 Direct Memory Access (DMA) channels, a 12 bit Analogue to Digital Converter (ADC) with 16 channels, a 12 bit Digital to Analogue (DA) converter, a hardware multiplier, 2 independent timers, 2 communication ports which can implement Universal Asynchronous Receiver Transmitter (UART), Serial Peripheral Interface (SPI) or Inter-Integrated Circuit

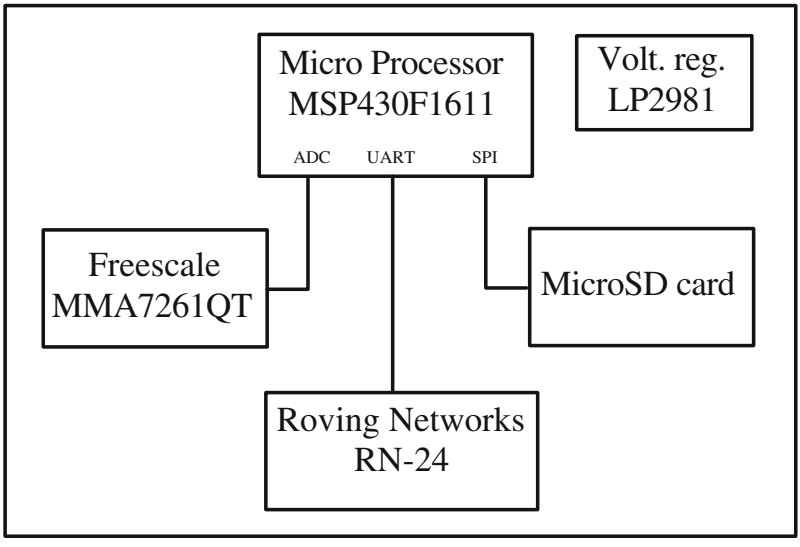


Fig. 1. Block diagram of fall and mobility sensor

(I2C) communication protocols and a watch dog. Due to this impressive selection of hardware interfaces and the availability of three DMA channels, it is possible to implement significant functionality without the need to have the core of the microprocessor running continuously.

3.2 Freescale Accelerometers

The Freescale MMA7261QT tri-axial accelerometer is a low cost accelerometer with selectable outputs between $\pm 2.5 - 10g$[1]. The principle of operation is based on sensing changes in capacity due to flexing of moveable beams under the influence of accelerations. With a power consumption of $500\mu A$, the sensor is ideally suited for low-power applications. The measured accelerations are filtered using on-board single pole switched capacitor filters and can be directly fed to an AD converter.

3.3 Bluetooth Module

The Roving Networks RN-24 can be obtained as a class 1 or 2 module on a small Printed Circuit Board (PCB), which allows for through-hole, and thus easy, integration on prototype PCB's. The module offers sustainable data rates of up to 100 kb/s at transmit/receive powers of around 30-50 mA with which it is by far the biggest consumer on the board. Power consumption can be minimized by using appropriate sleep modes. The RN-24 was chosen

[1] g Is the gravitational constant in $\frac{m}{s^2}$.

for its ease of integration in prototype hardware, but it should be noted that any radio with a UART or similar interface could be used. The use of a Zigbee radio is for example appealing from an energy efficiency aspect, but introduces challenges due to the relatively limited number of available Zigbee interfaces for PCs and mobile phones.

3.4 Power Requirements

Due to the use of a low power microprocessor with the ability to turn off peripherals and thus lower power consumption further, a low energy sensor can be obtained. The potential for this power reduction is determined by the functional requirements of the sensor. To indicate a ceiling for the energy consumption, the power requirements of the sensor without explicit power reducing measures is here discussed. In this scenario, the processor core and accelerometers are powered continuously. This results in a quiescent current of $370\mu A$ at 3.3V. However, whilst communicating the sensor consumes 38.4 mA and this clearly inhibits long term autonomous use of this sensor. For the prototype of this platform a rather large Lithium Ion battery with capacity of 1000mAh was chosen. With this battery it is possible to transmit mobility data constantly for periods exceeding 26 hours. For monitoring applications, in which mobility data will normally only be stored locally and only urgent information is relayed using Bluetooth, the battery can be chosen far smaller. Although the cost of communication is likely to drop significantly with the imminent use of Bluetooth Low Energy, Bluetooth is a rather power hungry communication channel. Its widespread availability makes Bluetooth the ideal wireless technology for clinical use of the sensor. For stand-alone applications a low-power wireless technology, such as Zigbee (802.15.4) or a proprietary technology may be more appropriate.

3.5 Communication

Although the Bluetooth standard incorporates methods for guaranteed message delivery through resending of messages, the respective functionality is normally not exposed to the designer. Moreover, the exact implementation is dependant on the drivers used and as a result message receptions can be quite unpredictable at times (on less powerful platforms messages can get 'lost' in the stack or buffers can overflow). In addition to these issues, there exist different ways of making the identity of the sender and receiver known. The Bluetooth standards specify 'services' that can be used to identify devices and Media Access Control (MAC) addresses can also be used. Again, due to the various abstraction levels in the software, it may not always be straight forward to identify an arbitrary Bluetooth radio in the software on the Bluetooth master device.

For these reasons a high level protocol with sender and receiver identification can be extremely useful. A message protocol was defined for bi-directional communication with the fall and mobility sensor. The payload

of the message, with a maximum length of 226 bytes[2], is preceded by a delimiter (0xFC) and 8 byte header and followed by a second delimiter (0xFD). As the second byte of the header contains the message type, the protocol allows for early decisions on whether or not to read a message. The number of bytes to be read is stored in the first field of the header. In combination with the delimiters, the message length is used to decide on the integrity of the received message prior to further processing. The integrity of the message is further tested using a checksum.

4 Fall Sensor Software

4.1 Embedded Software

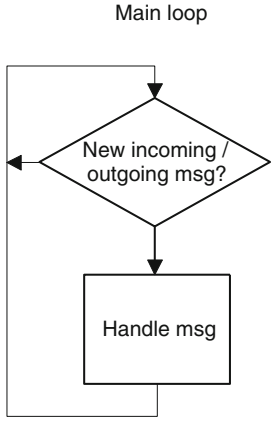
The fall and mobility sensing algorithm is largely interrupt driven. Figure 2a shows the main loop of the software, in which the software continuously checks for incoming and outgoing messages. The main loop of the program is periodically interrupted by various interrupts. Two interrupts occur at fixed intervals. The first of these interrupts is a timer interrupt which triggers a reading from the AD converter, as depicted in figure 2b. Upon finishing the conversion, the AD converter will send an interrupt indicating a new reading is ready for further processing, which will trigger the process depicted in figure 2e. First and foremost, in this interrupt handler the fall algorithm is implemented and used to analyze the latest data for high impacts and, if necessary, to start a further analysis into the origin of detected impacts. If event or alert messages result from this analysis, these are stored on a stack and a transmission will be initiated in the program's main loop. This is done by sending characters one by one to the UART (Universal Asynchronous Receiver/Transmitter). Once the UART is ready to accept the next character, it sends an interrupt request which is handled as depicted in figure 2d. In this interrupt routine the next character is fetched from the message buffer and sent to the UART until all characters have been sent. Receiving messages is performed in a similar fashion.

As depicted in figure 2c, upon reception of a character by the UART, the latter will send an interrupt request to the processor. The character is then transported from the UART to an incoming message buffer, after which the UART is ready to receive the next character.

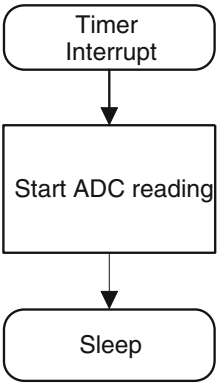
5 A Generic Sensor Interface Architecture

This section will detail the architecture used for a mobile interface to the sensor implemented on a Blackberry 'Storm' device. A modular system has been a key design goal as has attempting to keep the code base small and

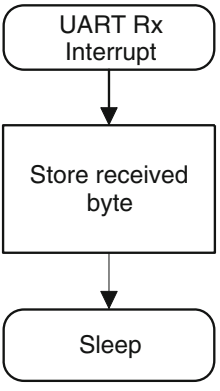
[2] The maximum payload size is determined by the wish to send DM5 packets in order to keep energy consumption to a minimum.



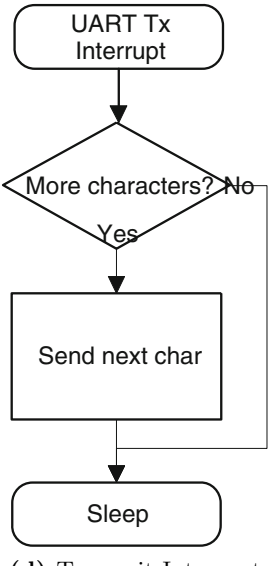
(a) Main loop of fall and mobility sensing software



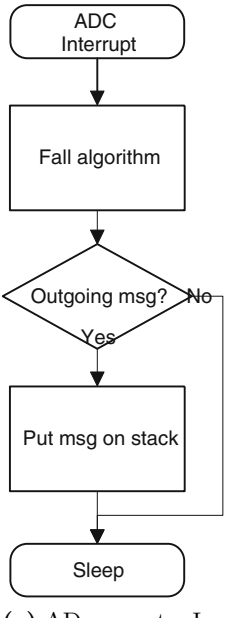
(b) Timer Interrupt



(c) Receive Interrupt



(d) Transmit Interrupt



(e) AD converter Interrupt

Fig. 2. Main software components of the fall and mobility sensor

efficient for use on resource constrained devices such as phones. The top level of the design consists of graphical user interfaces (GUI) and agents. The concept and function of agents will be further discussed after a discussion of the low level layers in the architecture.

In addition to the evident importance of reliable fall detection capabilities and the prompt relay of related messages to a caretaker or rescue services, an important aspect of fall and mobility monitoring is the feedback that such devices can give to the user. In many cases, the most effective means of preventing or overcoming mobility issues is frequent and sufficient exercise. The fall and mobility monitor can aid the user in complying with a set treatment programme by monitoring the user and suggesting further exercises to meet targets. This application clearly exercises one of the main benefits of using a mobile phone to interface with the sensors; the user is immediately provided with feedback which can improve treatment and this information is presented on a device that most, if not all, user groups are familiar with[3]. Caretakers and health professionals can access the data extracted from the users mobility patterns through any of the transport mechanisms the phone provides.

5.1 General Overview

A high-level overview of the generic sensor interface architecture is given in figure 3. The Bluetooth connection from the sensor is visible as a stream of bytes on the BlackBerry handset. The message protocol discussed in section 3.5 was used. For each sensor connection on the handset a state machine monitors the byte stream to detect when a valid message has been seen. The checksum of the message is computed, and if valid, a new SensorMessage object is created by the state machine. The state machine then resets its state ready to begin monitoring for a new message.

The SensorMessage object creates a SensorHeader object, which accesses the raw bytes stored in the SensorMessage and parses them into native data types. The SensorHeader also logs status information to the DataStore.

Using the message type information gleamed from the SensorHeader, the SensorMessage dispatches the remaining raw bytes to the appropriate message handler defined for the particular message type received. e.g. a mobility message contains information on the user's cadence so a MobilityMessage object parses the bytes according to the defined structure of a mobility message. Once parsed, these readings are stored in the DataStore.

The DataStore acts as the division between the lower and the higher layers of the interface. The lower layers need only send their readings to it. All records stored in the DataStore are interactive objects, which maintain their own value and a list of Agents who require notification when the value changes. Agents may register for updates on a record that does not yet exist, so that if the record should ever receive data from the lower layer, the agent will be immediately informed.

Agents monitor the records, such as 'step count' and may either present this data to the user in the form of a graphical screen (as in figure 4), store

[3] Phones with large touch displays are currently becoming increasingly popular. These phones offer accessible interfaces for less technologically experienced users or users with reduced motor skills.

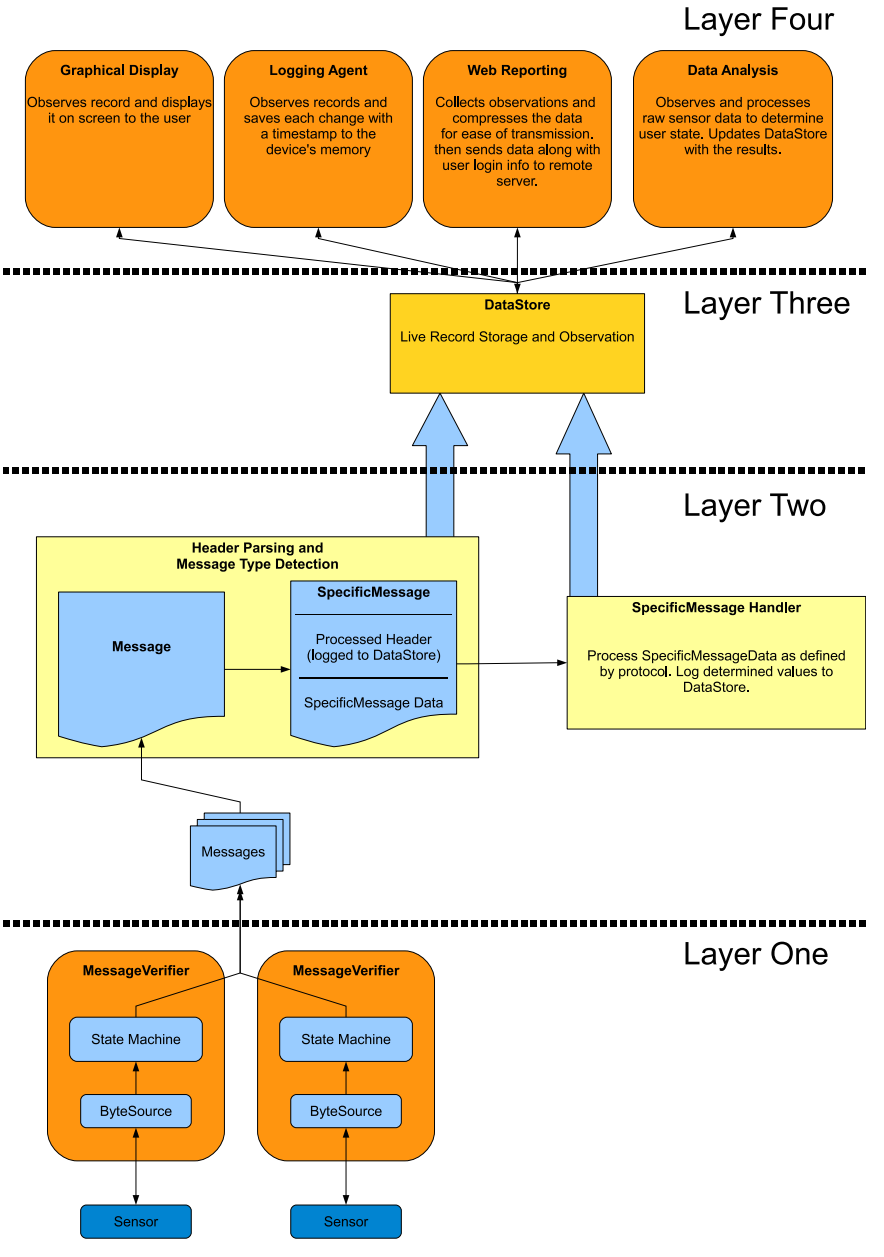


Fig. 3. High-level overview of generic sensor interface architecture

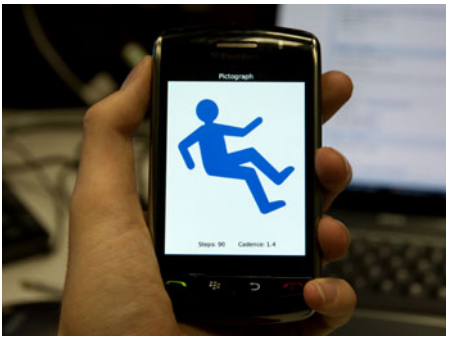


Fig. 4. Graphical Interface used to visualize information from a fall and mobility sensor

the data in the device's flash memory, upload the data to a remote server, or in the case of an emergency, make use of the handset's telephony features to directly contact emergency services.

5.2 ByteSources

At the bottom of the stack are the ByteSources. The function of this layer is to interact with whatever data source they are designed for, such that they may buffer data as it is received and push the individual bytes of the data one at a time into the next layer, the MessageVerifier. Each ByteSource is required to create its own instantiation of a MessageVerifier (described below). For reasons relating to the functionality of upper layers each ByteSource also passes its object (a.k.a.'this') reference into the constructor of its personal MessageVerifier.

5.3 MessageVerifier

The MessageVerifier class models a state machine that is responsible for ensuring that the bytes passed into it from a ByteSource match an expected sequence and conform to the defined message protocol. Should a message meet these requirements it is then checksummed to ensure that no errors occurred in transit. Internally, bytes are input in a piecemeal fashion strictly from a single ByteSource and are stored in memory that is managed by the MessageVerifier. The internal state machine monitors the number and contents of the received bytes and when it has determined that enough bytes have been received that they constitute a message, it performs the checksum. Should the message pass all of the tests the MessageVerifier creates a new instance of a SensorMessage passing into its constructor the valid message as well as a reference to the ByteSource that created the message. The MessageVerifier does not hold a reference to this new SensorMessage and once it

has been created the state machine will reset and be ready to begin processing the next message. This functionality is completely opaque to the ByteSource.

5.4 SensorHeader

Though the MessageVerifier passes a valid message into a SensorMessage constructor, the very first action of the SensorMessage constructor is to pass the same valid message into a SensorHeader, for this reason it will be described first. The SensorHeader is processed much like any of the other messages, the key difference being that header data is always present regardless of the message type being received.

Three features differentiate it from other messages, firstly it is responsible for comparing the sequence number of the message against the expected sequence number. If these numbers do not match, an error is logged in the DataStore. If they do match then the next expected sequence number is updated to reflect the current sequence number plus one.

Secondly, the SensorHeader is capable of logging detailed statistics on the frequency of messages of differing lengths and types. This is controlled through a toggle value in the DataStore.

Thirdly, once the SensorHeader has fully parsed the input and determined key values such as the message type and source ID of the Sensor, it stores these values in public access members, freeing other messages to deal only with their specific message content while still retaining (simplified) access to the data stored in the message header. The message type value in particular is required by a SensorMessage and is the reason the SensorHeader is created first.

5.5 SensorMessage

As described above, the first order of business for a SensorMessage is to create a new SensorHeader and store a reference to it. Once the SensorHeader has been fully constructed, the SensorMessage can access the message type data and, using a switch statement, determine which of the various message types to create.

5.6 SensorRecords

SensorRecords represent the fundamental data storage unit within the DataStore. At a basic level, a SensorRecord may be essentially viewed as a large wrapper for a single integer value. The most useful aspect of a SensorRecord is that it maintains a list of objects that implement the Observer interface and provides methods for additional Observer-implementing objects to be appended to this list. Classes that implement Observer provide a public access method called `update()` that takes an Observable object as a parameter. SensorRecord extends the Observable class such that when a SensorRecord

receives a new integer value to store (handled by the DataStore), should it decide that it has changed enough to warrant notifying the objects that are observing it, it will iterate over its list of Observers calling the `update()` method on each of the elements, passing itself as the only argument. The Observers can use the accessor methods defined in SensorRecord on the `this` reference they receive to find out exactly which SensorRecord it was that triggered the update. This is necessary as the same Observer may be observing a multitude of Observable objects yet each of them will call the same `update()` method of the Observer.

5.7 The DataStore

The DataStore contains only static members and methods, it is always available and no other part of the system is responsible for instantiating it. At its heart is a hash table with Strings as keys and SensorRecords as entries. DataStore provides some simple accessor methods that allow client classes to directly access a SensorRecord (`getRecord(String key)`) or to act as a proxy so that storing or retrieving the integer value associated with a particular SensorRecord is made easier due to the reduced type casting (the `Store(String key, int value)` method and the `retrieve(String key)` method).

For each of the accessor methods, DataStore will first scan the hash table to determine if the record being queried already exists. Instead of returning an error in the case of a non-existent record, DataStore will automatically create the record with the default value of zero and will carry out the rest of the request on the newly created record as normal, e.g. if the store method was called it will change the value in the SensorRecord from the default, to the specified value. The reasoning for this behaviour is an attempt to reduce the coupling between agents and SensorRecords. Should an agent attempt to access a SensorRecord that had not yet been created because the message type that creates it has yet to be received (e.g. 'Fall Type' from 'FallAlertMessage') then there would be no clear mechanism for the agent to be notified should the record be created later in the session (polling is too inefficient to consider). By registering on a non-existent record the agent is able to ensure that should this record ever receive an update, it will be alerted through its `update()` method.

5.8 Records

'SensorRecords' are self contained objects that can store an integer value and are observable. 'Records' is a single class filled with static nested classes filled with either more static nested classes or static final Strings. The reason for this seemingly mindless class is that it serves as an invaluable programming aid when accessing particular records from the DataStore. Due to the nature of the DataStore, if a non-existent SensorRecord is accessed, the DataStore

will create the record with default values and return it, this is done so that agents can register on SensorRecords that have not yet had data stored in them but are expected to do so.

The problem arises that if a programmer makes a typographical error in accessing the DataStore, there will be no indication that the record they are requesting will never receive an update. By categorising the expected records into a hierarchical system, errors such as this are averted and record selection while programming is made more intuitive.

5.9 ConfigDB

Storing observable records that represent the current state of a Sensor has immediate usefulness, but the limitation on what data can be stored (integers only) has made it difficult to use the DataStore to store anything else, such as the list of currently connected sensors or the set of user defined options that control the interface itself. For this reason a separate section of the Records tree was created which does not have the requirement to return SensorRecords when queried. In implementation, it is an entirely separate hash table to the one that is used for the SensorRecords but is still maintained by the DataStore. The ConfigDB does not allow new entries to be created dynamically as the regular DataStore does, if an attempt is made to access a record that does not exist then an exception is thrown. Valid records are listed under the 'config' tree in the Records file.

SourceList

The SourceList stores the list of all Sensor source IDs that have been seen by the system along with a reference to the ByteSource that is responsible for it. In SensorMessage, after the header processing but before the dispatch to the message handlers, the source ID of the message is sent to the SourceList. The SourceList is designed for agents that monitor records and may find it necessary to transmit data back to the Sensor that generated the record. Normally, the only information about the Sensor available to the Agent is the SourceID from the top of the record name but with the SourceList the agent can look up the reference to the ByteSource responsible for handling that Sensor and call its `sendToSensor` method. This functionality would be used for example for the delivery of acknowledgement messages to the fall sensor.

As SourceList extends the Observable class, agents may also register as Observers of it so that they are notified when a new Sensor becomes connected to the device. In this way an agent may determine the source ID of the new sensor and immediately begin observing records of interest that it generates, or is expected to generate.

5.10 Agents

Agents are applications that interact with the contents of the DataStore, and, based on what they find or determine through calculation, will initiate some action or write some value back to the DataStore. Agents are self-contained and conduct all of their communication through the DataStore (i.e. an agent may observe the output record of another agent). Many agents may be 'running' simultaneously and each may observe any number of SensorRecords or other observable objects.

Agents represent the top layer of the component stack and offer a flexible way to expand the functionality of the interface. All agents extend the 'Agent' abstract class which exposes start and stop control to other aspects of the system. Also provided is a wrapper function that eases the act of registering as an observer of an object; this wrapper maintains a list of Observables that the agent has registered with so that if the agent is stopped the Observables will have the agent's entry removed from their update lists.

An example of an agent for the fall sensor interface is the FallHandler agent which monitors the `status.falldetected` record on a number of Sensor IDs and uses the BlackBerry OS integration features to place a phone call to a number defined in an Options file. A further example is the FallAlertAcknowledger which implements a protocol requirement that the Sensor receive an acknowledgement message after alerting the Handler to a fall event. Both agents also observe the SourceList and register themselves on the appropriate records anytime a new source ID becomes known to the interface.

Agents may also be used to log data to external media, to interact with a Web service, to compute user mobility patterns based on Sensor data, or implement virtually any other high-level functionality. The strength of the agent based approach is demonstrated by the potential for an agent to reprogram a Sensor with a new version of its firmware should one become available. This could for example be done by the web service agent, who can download the firmware to the device and signal the reprogramming agent which will or will not run as determined by user preferences.

Further examples of useful agents would be a high level 'agent manager' which would determine which agents to start at device boot time or application start time and a 'connection manager' agent that would ensure the Sensors are continuously connected and alert the user if a connection issue exists.

5.11 Graphical Interfaces

Graphical interfaces are implemented as a special form of agent. Their main goal is to access the DataStore and represent the information found there in a visual manner to the user. As they may observe any record, the graphical interface dynamically updates anytime new information is received from the sensor. User modifiable preferences are also exposed through a graphical agent.

6 Case Study: The CAALYX Project

In the recently successfully concluded CAALYX project (Complete Ambient Assisted Living eXperiment) funded by the European Community under the 6^{th} Framework Programme, fall and mobility monitoring played an important role. The aim of the CAALYX project was to provide a suite of sensors for healthy elderly users to detect potential health threats at an early stage. This section will detail how the integration of the fall and mobility sensor in this platform was performed.

6.1 Project Description

As mentioned above, the aim of the project was to detect potential health threats in elderly citizens at an early stage. This approach can help to significantly reduce hospitalization and thus the cost associated with health care provision. A detailed investigation of the most dominant health issues encountered by the elderly, led to the choice of a suite of sensors. The chosen sensors were for blood pressure, weight, temperature, blood oxygen saturation, pulmonary rate, cardiac information (heart rate and full ECG) and fall and mobility information. The mobility information obtained from the fall and mobility monitor played an important role in determining whether measurements were valid (movement artifacts play a dominating role in e.g. ECGs) and to schedule measurements (measurements were scheduled to coinside with expected periods of user inactivity or rest). For this reason, an intricate integration of this sensor in the platform and its data processing functionality, was required.

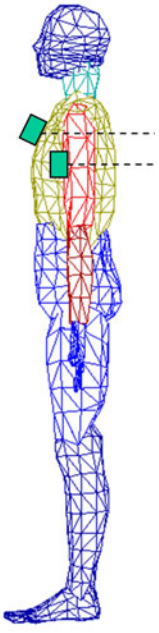
Trials were performed in two phases to allow for gradual integration of the fall and mobility hardware and functionality in the wider system. Both phases will be discussed separately.

6.2 Phase 1 Trials Description

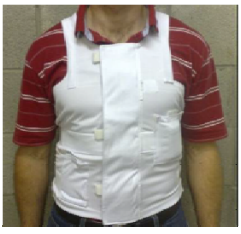
During Phase 1 of the CAALYX trials, a stand-alone fall and mobility sensor was used to reduce physical integration efforts and focus on system integration first to demonstrate the significant value of mobility information in a remote health monitoring system.

Sensor Location

In previous research [39], positive results were obtained with the fall sensor positioned on the chest as depicted in figure 5a. Hence this position was initially used to test the new fall sensor platform. After positive results the sensor was also used under the left arm, thus minimizing the influence of the device on the user. To achieve greater compliance with wearing the fall and mobility sensor located on the chest, the vest depicted in figure 5b was



(a) Sensor positioning



(b) Sensor Vest worn by Researcher

(c) Sensor Vest with Sensor near sternum location

Fig. 5. Details regarding Phase 1 sensor positioning and fixation

developed as an alternative to the chest strap used in the previous study [39]. The CAALYX Fall and Mobility Monitoring Vest is a light weight garment (230g) with a zipper on the front for ease of donning and doffing of the garment. Support elastic is incorporated into the vest to ensure the sensor is held close to the body. As can be seen in figure 5c the sensor is attached to the vest using velcro and covered with a flap of material which is also closed using velcro. A pocket is located at hip level for a mobile phone which can be used for communication with the sensor. A set of vests adjustable from sizes 2XS to 3XL was designed for the trials. The size adjustment was achieved using 2 velcro adjustment strips at each side (4 in total per vest). These should be adjusted so that no gap between the subjects body and the vest existed, but should not be so tight as to restrict breathing.

Phase 1 Fall Sensor Hardware

The CAALYX system setup during the phase 1 trials dictated a stand-alone fall and mobility sensor that communicated with the wider system through a Nokia N95 mobile phone. To increase robustness of the system as a whole, the fall and mobility sensor was equipped with a 1000mAh LiIon battery. This allowed monitoring of the user including on-board high resolution recording